

kdc-setup

이 문서에서 다루는 내용

- config-server가 FARM AD 계정, 사용자 keytab, 노드 ccache, Kerberos NFS 홈에 쓰는 흐름 전체
- 이 묶음 안에서 설계, 운영, 설정 문서를 어떤 순서로 읽으면 되는지
- `system/kerberos-nfs` 문서와 여기 문서의 경계

빠른 이동

필요할 때	문서
파일과 단계의 설계를 본다	설계
점검과 장애 대응 순서를 본다	운영
Helm, CI, Secret 값의 위치를 본다	설정
NAS와 NFS 쪽 Kerberos 설정을 본다	System Kerberos/NFS

`kdc-setup`은 config-server가 사용자 Pod를 만들 때 쓰는 Kerberos 설정 문서다. NAS service keytab, NAS KVNO, 다른 컨테이너 환경의 Kerberos 설정은 [System Kerberos/NFS](#)에서 다룬다.

문서 안내

문서	읽어야 하는 경우	핵심 내용
현재 페이지	문서의 역할을 먼저 볼 때	AD, Secret, 노드, Pod가 이어지는 순서
설계	계정과 Pod를 만들 때 어떤 파일이 필요한지 볼 때	AD, Secret, 선택 노드, ccache, NFS
운영	점검, 장애 대응, 노드 추가를 할 때	확인 순서와 명령
설정	Helm, CI, Secret 설정을 바꿀 때	값이 들어가는 곳과 확인 방법

계정부터 Pod까지의 순서

승인 시스템

- > config-server
- > FARM AD DC: 사용자·전용 그룹·RFC2307·keytab
- > Kubernetes Secret: 사용자 keytab 보관
- > 선택된 FARM 노드: root-only keytab·refresh timer·ccache
- > 사용자 Pod: keytab 없이 ccache와 FARM 홈을 사용

누가 무엇을 하나

- config-server는 계정과 Pod 생성·삭제를 요청하고 AD와 노드 관리 명령을 실행한다.
- AD는 사용자, 전용 그룹, RFC2307 UID/GID, 사용자 keytab을 만든다.
- 선택된 노드는 keytab을 root만 읽을 수 있는 경로에 두고 사용자 ccache를 갱신한다.
- Pod는 keytab을 받지 않고 ccache만 사용한다.
- NAS 홈 디렉터리 생성·삭제는 NAS 관리 경로에서 처리한다.
- 비밀 값과 변경 이력은 관리자 전용 시크릿과 배포 설정 문서에만 둔다.

이전 Pod 처리

새 계정과 새 Pod는 FARM AD 설정을 사용한다. 전환 전에 만든 Pod는 다시 만들기 전까지 이전 Realm이나 홈 마운트를 쓸 수 있다. 이전 Pod에 새 keytab이나 timer 파일을 덮어쓰지 말고, 현재 Pod의 환경 변수와 hostPath를 확인한 뒤 다시 만든다.

관련 문서

- [System Kerberos/NFS](#): NAS와 NFS의 Kerberos 설정
- [System container-images](#): 이미지가 ccache를 쓰는 방식
- [설정](#): config-server 배포 설정과 확인 방법

kdc-setup 설계

이 문서에서 다루는 내용

- AD 사용자, Secret, 노드 keytab, ccache가 어디에 놓이고 누가 쓰는지
- 계정 생성, Pod 시작, Pod·계정 삭제 때 Kerberos 관련 파일이 어떻게 바뀌는지
- 바꾸면 안 되는 보안·운영 규칙

빠른 이동

필요할 때	문서
전체 흐름을 먼저 본다	개요
점검과 장애 대응을 본다	운영
실제 설정값의 위치를 본다	설정

1. 이 설정에서 지킬 것

- FARM AD의 RFC2307 UID/GID와 NAS의 숫자 소유권을 같은 값으로 유지한다.
- 사용자 keytab은 Pod에 전달하지 않고, 선택 노드의 root 전용 경로에만 둔다.
- Pod를 실행하기 전에 해당 노드에서 사용자 TGT와 ccache 소유권을 검증한다.
- AD를 관리하는 접속과 노드를 관리하는 접속에 같은 키를 쓰지 않는다.
- NFS에서 D-state가 보이면 mount나 서비스를 반복해서 다시 시작하지 않는다.

2. 파일과 서비스가 있는 곳

```
config-server
├── AD 관리 채널 -> AD DC
│   └── 사용자 + <user>_gid + RFC2307 UID/GID + 사용자 keytab
├── Kubernetes API -> Secret krb5-keytab-<user>
└── 노드 관리 채널 -> Pod가 배치될 FARM 노드 한 대
    ├── /etc/decs-krb/keytabs/<user>.keytab    root-only
    ├── decs-krb-refresh@<user>.timer
    ├── /run/user/<uid>/krb5cc_aialab          사용자 소유
    └── Pod는 ccache와 FARM 홈만 공유
```

AD 사용자와 Secret은 계정이 있는 동안 유지한다. 노드의 keytab, 환경 파일, timer, ccache는 Pod가 올라간 노드에 서만 쓴다. Pod를 지우면 노드의 파일은 지우지만 AD 사용자와 Secret은 다음 Pod를 위해 남긴다.

3. 계정과 인증 파일

항목	위치	사용하는 곳	권한
사용자·전용 그룹	FARM AD	config-server, winbind, NAS	RFC2307 UID/GID 일치
사용자 keytab	Kubernetes Secret	config-server	관리자만 관리
노드 keytab	선택된 FARM 노드	root, refresh service	root 소유 0400

항목	위치	사용하는 곳	권한
갱신 환경 파일	선택된 FARM 노드	refresh service	root 소유 0400
사용자 ccache	/run/user/<uid>/krb5cc_ailab	Pod, 호스트 NFS 경로	사용자 UID/GID, 0600
머신 ccache	/tmp/krb5cc_0	rpc.gssd	현재 노드 머신 principal 전용

/tmp/krb5cc_0에는 현재 노드의 머신 ticket만 둔다. 여기에 사용자 ticket이나 다른 Realm ticket을 넣으면 NFS 인증이 달라질 수 있다. 사용자 확인과 복구에는 별도 ccache를 사용한다.

4. 계정 생성부터 삭제까지

계정 생성

1. config-server가 UID/GID를 할당하고 계정 파일을 갱신한다.
2. NAS 관리 경로가 같은 숫자 소유권으로 홈 디렉터리를 준비한다.
3. AD 관리 채널이 사용자와 <user>_gid 그룹을 만들고 RFC2307 속성을 설정한다.
4. AD가 사용자 keytab을 발급하고 config-server가 Kubernetes Secret에 저장한다.

마지막 단계가 실패하면 계정 생성도 실패한다. NAS 홈과 계정 파일은 되돌리려고 하지만, AD 사용자·전용 그룹·Secret 일부가 남을 수 있다. 같은 이름으로 다시 만들기 전에는 이 세 곳을 확인한다.

Pod 시작

1. config-server가 대상 FARM 노드를 선택한다.
2. Secret의 keytab을 해당 노드의 제한된 관리 채널로 전달한다.
3. 노드는 root 전용 keytab·환경 파일·decs-krb-refresh@<user>.timer를 준비한다.
4. refresh service가 사용자 TGT를 발급하고 ccache 소유권을 검증한다.
5. 검증에 성공할 때만 Pod가 ccache와 FARM 홈을 hostPath로 공유하며 시작한다.

Pod·계정 삭제

Pod 삭제는 해당 노드의 timer, keytab, 환경 파일과 ccache를 정리한다. 계정 삭제는 AD 사용자와 Secret을 지우고 모든 FARM 노드에 사용자 keytab, timer, ccache 정리를 요청한다. 전용 그룹은 자동으로 지우지 않는다. 지울 필요가 있으면 운영 규칙을 확인한 뒤 따로 처리한다.

5. 접속 경로

접속 경로	대상	할 수 있는 일
AD 관리 채널	AD DC	사용자·전용 그룹 생성/삭제, keytab 발급
노드 관리 채널	FARM 실행 노드	keytab 배포·삭제, timer 제어, ccache 검증

접속 경로	대상	할 수 있는 일
NAS 관리 채널	NAS	홈 생성·소유권 설정·삭제

각 접속 경로는 다른 키와 forced-command를 사용한다. AD를 수정하는 키로 모든 노드에 일반 셸 접속을 할 수 없게 한다.

6. 바꾸면 안 되는 규칙

1. 사용자 keytab을 Pod에 복사하거나 mount하지 않는다.
2. AD의 UID/GID, config-server 계정 파일, NAS 소유권을 따로 바꾸지 않는다.
3. Pod 시작 전에 대상 노드의 TGT와 ccache 소유권을 확인한다.
4. 머신 ccache에는 현재 노드의 머신 principal만 둔다.
5. D-state가 있는 노드에서 강제 unmount·반복 mount·반복 service restart를 하지 않는다.

kdc-setup 운영

이 문서에서 다루는 내용

- AD, Secret, 노드 keytab, ccache, NFS 상태를 어떤 순서로 확인할지
- 계정과 Pod 생성·삭제 뒤에 어떤 흔적이 남아야 하고 무엇이 지워져야 하는지
- 사용자 단위 장애와 노드 전체 장애를 어떻게 구분할지

빠른 이동

필요할 때	문서
전체 구조와 역할 분담을 먼저 본다	개요
파일 위치와 보안 규칙을 본다	설계
Helm, CI, Secret 값의 위치를 본다	설정

1. 작업할 때 지킬 것

1. 계정과 Pod 생성·삭제는 config-server API로 수행한다.
2. keytab, 개인키, Secret `data` 는 일반 운영 문서·로그·티켓에 출력하지 않는다.
3. 사용자 장애는 해당 사용자와 Pod가 배치된 노드 범위에서 먼저 확인한다.
4. `/tmp/krb5cc_0` 에는 현재 노드의 머신 principal 이외의 ticket을 발급하지 않는다.

5. NFS 관련 D-state가 있으면 반복 restart, 강제 unmount, 연속 mount를 중단한다.

2. 정기적으로 확인할 것

Kubernetes

```
kubectl -n ailab-infra get deploy,pod,cronjob
kubectl -n ailab-infra get events --sort-by=.lastTimestamp
kubectl -n ailab-infra logs deploy/containersssh-config-server --since=30m
```

config-server가 Ready인지, AD 계정 생성과 FARM 노드 설정이 반복해서 실패하지 않는지 확인한다. 사용자 keytab Secret은 값이 아니라 메타데이터만 확인한다.

```
target_user='<user>'
kubectl -n ailab-infra get secret "krb5-keytab-$target_user" \
-o custom-columns=NAME:.metadata.name,CREATED:.metadata.creationTimestamp
```

AD와 노드 인증 파일

AD DC에서 사용자·전용 그룹·RFC2307 번호를 확인하고, 선택 노드에서는 timer와 ccache의 권한·유효성을 확인한다.

```
target_user='<user>'
target_uid='<uid>'

sudo samba-tool user show "$target_user"
sudo samba-tool group show "${target_user}_gid"
sudo systemctl is-enabled "decs-krb-refresh@$target_user.timer"
sudo systemctl is-active "decs-krb-refresh@$target_user.timer"
sudo stat -c '%U %G %a %n' \
    "/etc/decs-krb/keytabs/$target_user.keytab" \
    "/etc/decs-krb/refresh.d/$target_user.env" \
    "/run/user/$target_uid/krb5cc_ailab"
sudo env KRB5CCNAME="FILE:/run/user/$target_uid/krb5cc_ailab" klist
```

keytab·환경 파일은 root 소유 0400, ccache는 대상 UID/GID 소유 0600 이어야 한다. klist -k 로 keytab 내용을 출력하지 않는다.

NFS와 노드 Kerberos ticket

```
systemctl is-active rpc-gssd
findmnt -T /home/tako2/share/user -o TARGET,SOURCE,FSTYPE,OPTIONS
sudo klist -c FILE:/tmp/krb5cc_0
ps -eo pid,stat,comm,args | awk '$2 ~ /^D/'
```

홈 경로 mount, rpc.gssd, 머신 principal, D-state를 함께 확인한다. Pod에서만 문제가 나면 Pod UID/GID와 ccache 공유 경로를 먼저 본다. 노드 전체에서 문제가 나면 머신 ticket, rpc.gssd, NFS mount를 먼저 본다.

3. 계정과 Pod를 만든 뒤 확인할 것

계정 생성 후

1. config-server 계정 파일과 AD RFC2307 UID/GID를 대조한다.
2. Secret 메타데이터, NAS 홈의 숫자 소유권·모드를 확인한다.
3. Pod 생성 뒤 대상 노드의 timer·ccache와 Pod 내부 홈 읽기·쓰기를 확인한다.

Pod 삭제 후

대상 노드에서 timer, keytab, 환경 파일이 제거됐는지 확인한다. AD 사용자와 Secret은 다음 Pod 실행을 위해 남아야 한다.

```
target_user='<user>'
sudo systemctl status "decs-krb-refresh@$target_user.timer" --no-pager
sudo test ! -e "/etc/decs-krb/keytabs/$target_user.keytab"
sudo test ! -e "/etc/decs-krb/refresh.d/$target_user.env"
```

계정 삭제 후

AD 사용자와 사용자 keytab Secret이 사라졌는지, 모든 FARM 노드의 사용자 timer·keytab·ccache가 정리됐는지 확인한다. 전용 AD 그룹은 자동 삭제 대상이 아니므로 임의로 지우지 않는다.

4. 문제별 확인 순서

증상	먼저 확인할 것	금지할 것
AD 생성·keytab 발급 실패	config-server stage, 대상 DC, 동일 이름의 사용자·그룹	DC wrapper의 반복 직접 실행
대상 노드 배포 실패	노드 목록, 제한된 SSH 채널, keytab 파일 모드, service journal	keytab을 복사해 우회
ccache 갱신 실패	DNS·시간·Realm·principal·keytab 모드	keytab 내용을 출력

증상	먼저 확인할 것	금지할 것
Pod 홈 접근 실패	Pod UID/GID, ccache, rpc.gssd, 머신 ticket, mount	NAS를 바로 재시작
NFS D-state	stack·kernel log·NFS 응답·네트워크	반복 mount/unmount 또는 service restart

머신 ccache의 principal이 잘못됐거나 ticket이 만료된 경우에만, 정확한 현재 노드 머신 principal을 확인한 후 별도 승인 절차에 따라 복구한다. 사용자 principal이나 다른 Realm을 지정하지 않는다.

5. 노드 추가·변경

신규 AD DC와 FARM 실행 노드는 먼저 제한된 관리 채널, 시간 동기화, rpc.gssd와 FARM 홈 mount를 준비한다. 시험 노드 한 대에서 시험 사용자로 keytab 권한, timer, ccache, TGT, NFS 읽기·쓰기를 검증한 뒤 노드 목록에 추가한다.

공통 refresh 스크립트·systemd 템플릿의 변경은 사용자 배포 중에도 다시 설치될 수 있다. 시험 노드에서 확인한 뒤 기존 노드의 버전·소유권을 비교하고 확대한다.

kdc-setup 설정

이 문서에서 다루는 내용

- FARM AD 관련 설정이 Helm values, CI secret, Kubernetes Secret, 노드 파일 중 어디에 들어가는지
- 값을 직접 노출하지 않고 배포 상태를 확인하는 방법
- 설정을 바꾸기 전에 범위를 어떻게 나눠 확인해야 하는지

빠른 이동

필요할 때	문서
전체 흐름과 문서 경계를 먼저 본다	개요
파일·서비스 위치와 보안 규칙을 본다	설계
운영 중 점검 순서를 본다	운영

설정이 들어가는 곳

config-server의 FARM AD 설정은 Helm values, CI secret, Kubernetes Secret, 노드의 root 전용 파일에 나뉘어 있다. 실제 비밀 값과 변경 이력은 관리자 전용 시크릿과 배포 설정 문서에만 두며, 이 문서에는 값, 개인키, keytab을 적지 않는다.

CI Secret / 관리자 전용 시크릿 문서

- > Helm values 주입
- > config-server Deployment 환경 변수·Secret mount
- > 제한된 AD/노드 관리 채널
- > 노드 root-only keytab과 사용자 ccache

설정별 사용처

구분	대표 항목	사용하는 곳	확인할 것
Realm	KRB5_REALM	config-server, 사용자 Pod	새 설정은 FARM AD Realm을 쓴다.
노드 관리	FARM_SSH_*, 노드 목록	config-server	AD 관리용 키와 계정을 섞지 않는다.
AD 관리	FARM_AD_SSH_*, DC 목록	config-server	AD DC에서 허용한 명령만 실행한다.
홈 경로	FARM_HOME_MOUNT_ROOT	사용자 Pod	FARM 노드에 이미 마운트된 NFS 홈 경로를 가리킨다.
NAS 홈 관리	NAS SSH 설정	config-server	홈 디렉터리 생성과 삭제에만 쓴다.
사용자 keytab	krb5-keytab-<user> Secret	config-server	Pod에는 마운트하지 않는다.

Kubernetes 배포 확인

값을 출력하지 않고 Deployment가 필요한 환경 변수와 Secret mount를 참조하는지 확인한다.

```
kubectl -n ailab-infra get deploy containersssh-config-server \
-o custom-columns=IMAGE:.spec.template.spec.containers[0].image,SA:.spec.template.spec.serviceAccountName
kubectl -n ailab-infra describe deploy containersssh-config-server
kubectl -n ailab-infra get secret farm-ssh-key farm-ad-ssh-key
```

사용자 keytab Secret은 이름·생성 시각만 확인한다. `kubectl get secret -o yaml`, `jsonpath`로 `data`를 출력하거나 terminal history에 남기는 명령은 사용하지 않는다.

노드에서 확인할 파일

노드는 사용자별 keytab을 `/etc/decs-krb/keytabs/` 아래 root 전용으로 보관하고, `decs-krb-refresh@<user>.timer`가 사용자 ccache를 갱신하도록 준비한다. 노드 머신 ccache와 사용자 ccache는 서로 다른 파일·principal로 분리한다.

대상	설정 기준
사용자 keytab·환경 파일	root 소유, 0400

대상	설정 기준
사용자 ccache	대상 UID/GID 소유, 0600
머신 ccache	현재 노드 머신 principal만 사용
rpc.gssd	노드 Kerberos NFS 경로와 함께 점검
forced-command 계정	일반 셀·포괄적인 sudo 없이 허용 동작만 실행

아직 쓰지 않는 ConfigMap

config-server에 마운트되는 `krb5-conf` ConfigMap은 현재 계정과 Pod 생성에 직접 쓰는 Kerberos 설정 파일이 아닙니다. config-server가 KDC 명령을 직접 쓰게 되면 KDC와 admin server 주소를 어디에서 넣을지 먼저 정한다. 그 전에는 비어 있는 endpoint를 정상 설정으로 보거나 복구에 사용하지 않는다.

설정을 바꾸기 전에 확인할 것

1. AD 관리, 노드 관리, NAS 홈 관리 중 무엇을 바꾸는지 먼저 정한다.
2. 실제 비밀 값은 관리자 전용 시크릿 문서에서만 확인하고 일반 문서에 복사하지 않는다.
3. 시험 노드 한 대와 시험 사용자로 ccache·NFS 접근을 확인한다.
4. 기존 Pod가 이전 Realm·홈 경로를 쓰는지 확인하고, 설정값을 덮어쓰지 않는다.
5. 검증 뒤에만 CI/Helm 설정을 확대 적용한다.